

图应用
双联通分量：判定准则

关节点 + 双连通分量

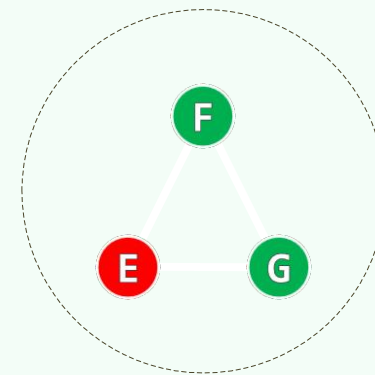
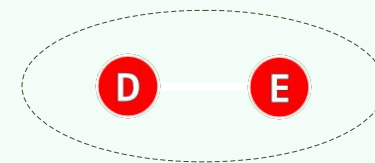
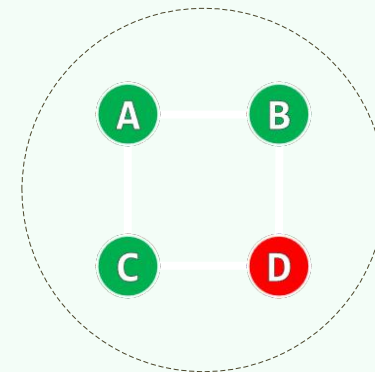
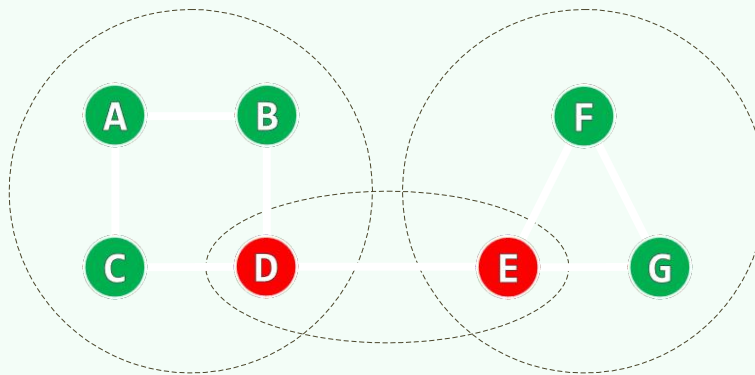
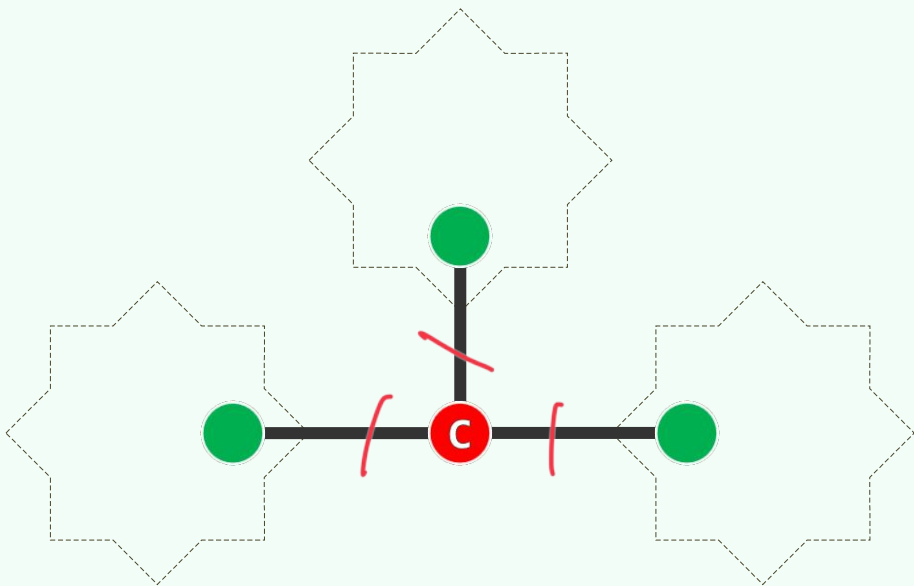
关键节点

❖ **无向图**的关节点: //articulation point, cut-vertex

其删除之后, 原图的双连通分量增多 //connected components

❖ 无关节点的图, 称作双 (重) 连通图 //bi-connectivity

❖ 极大的双连通子图, 称作双连通分量 //Bi-Connected Components



Brute-Force

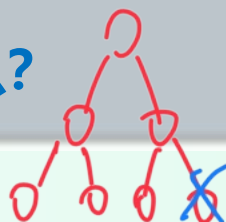
- (双连通分量)
- ❖ 给定无向图，如何确定各BCC?
 - ❖ 先考察简单的版本：如何确定关节点?
 - ❖ 蛮力：对每一顶点 v ，通过遍历检查 $G \setminus \{v\}$ 是否连通
 - ❖ 共需 $O(n * (n + e))$ 时间，太慢!

而且，即便找出关节点，各BCC仍需确定

- ❖ 改进：从任一顶点出发，构造DFS树

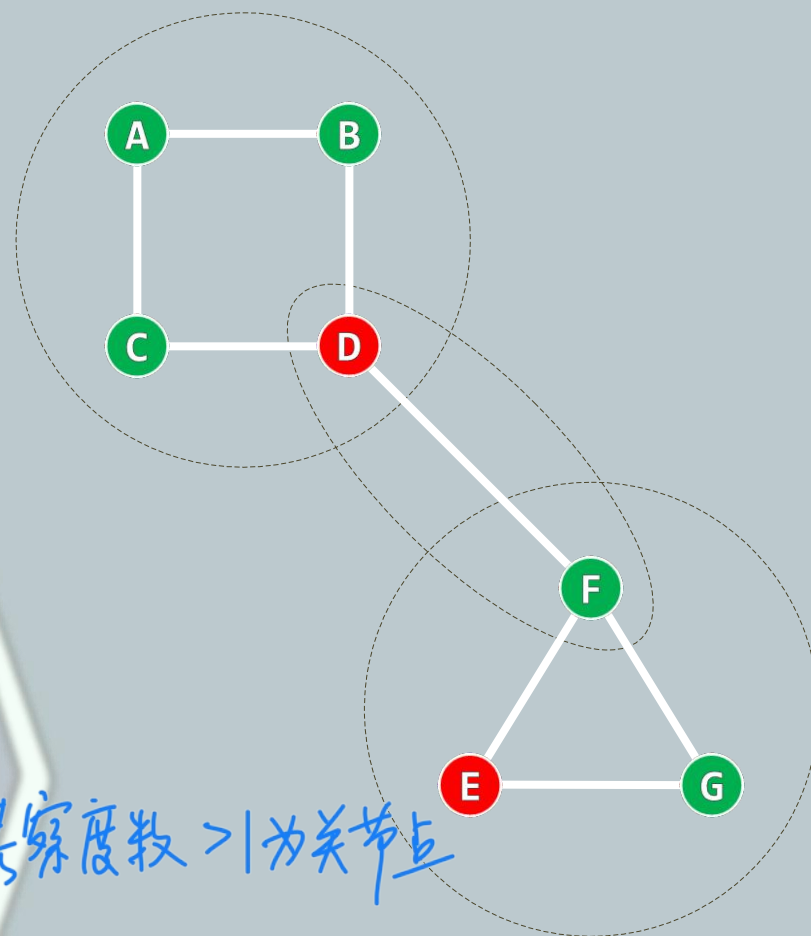
根据DFS留下的标记，甄别是否关节点

- ❖ 比如，叶节点绝不可能是关节点 //为什么?

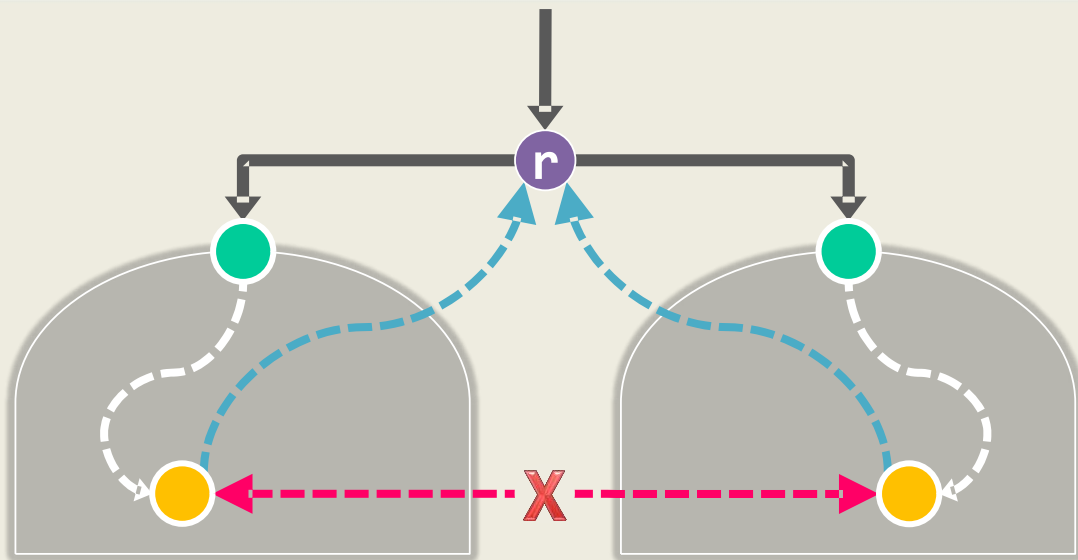


根需要考察度数 > 1 为关节点

去掉不影响连通性



非叶节点

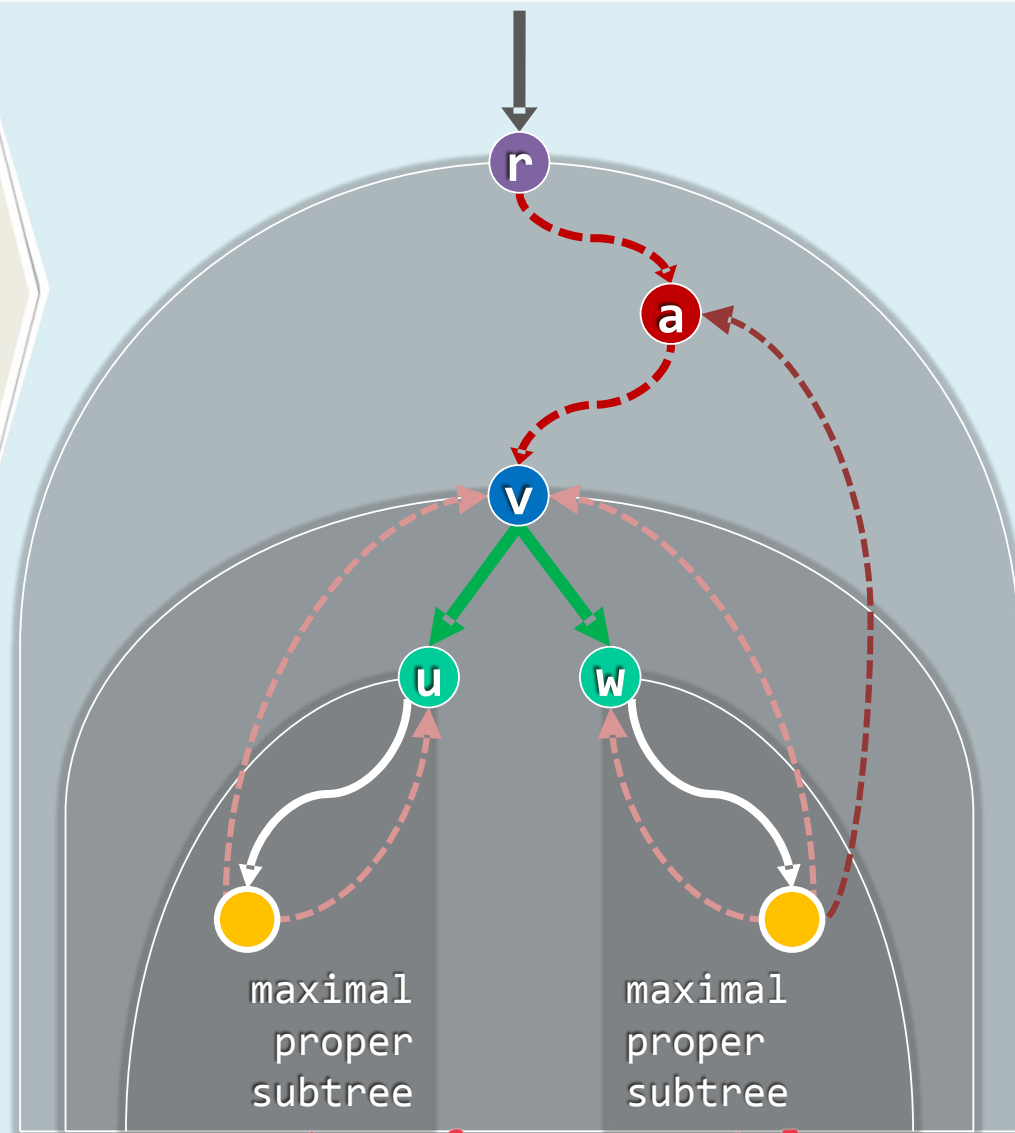


❖ 根r: 必须至少有2棵子树

❖ 内部节点v:

- 有某个孩子u, 而 $\text{subtree}(u)$ 不能经由BACKWARD边, 联接到v的任何真祖先a

❖ 此时, $\{v\} = \text{BCC}(u) \cap \text{BCC}(\text{parent}(v))$



对右侧, 去掉不连通, 右侧则连通

Highest Connected Ancestor (属性)

❖ $\text{hca}(v)$ = subtree(v)经后向边能抵达的最高祖先

❖ 由括号引理: dTime越小的祖先, 辈份越高 (若从 dtime 理解)

❖ DFS过程中, 一旦发现后向边(v, u)

即取: $\text{hca}(v) = \min(\text{hca}(v), \text{dTime}(u))$

❖ DFS(u)完成并返回 v 时

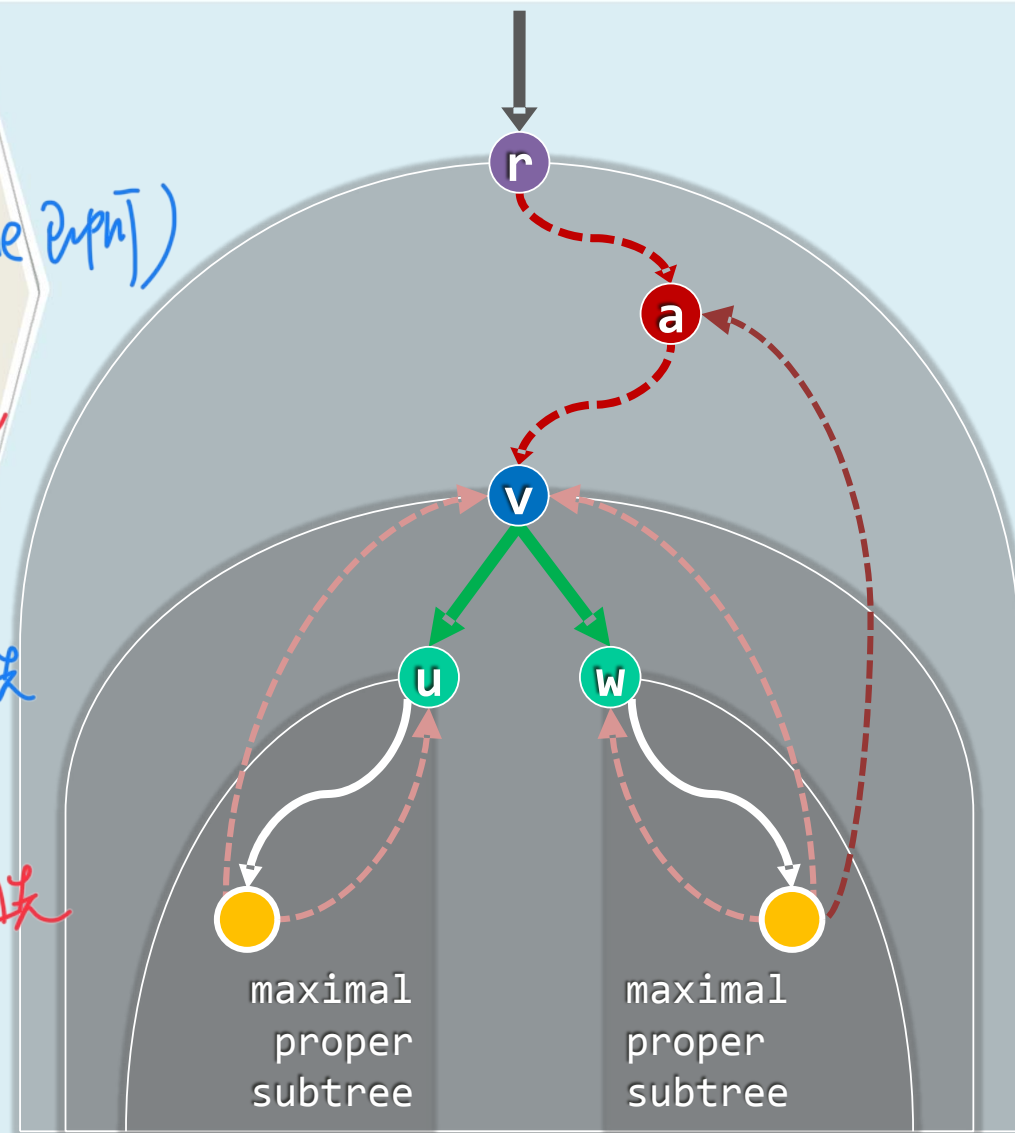
若有: $\text{hca}(u) < \text{dTime}(v) \Rightarrow u$ 可以到达更高祖先
同时更新 ✓

即取: $\text{hca}(v) = \min(\text{hca}(v), \text{hca}(u))$

v 可以通过孩子到达更高祖先

❖ 否则, 即可断定: v 系关节点, 且

$\{v\} + \text{subtree}(u)$ 即为一个 BCC



图应用
双联通分量：算法

Graph::BCC()

```
#define hca(x) ( fTime(x) ) //利用此处闲置的fTime

template <typename Tv, typename Te>

void Graph<Tv, Te>::BCC( Rank v, int & clock, Stack<Rank> & S ) {

    hca(v) = dTime(v) = ++clock; status(v) = DISCOVERED; S.push(v);

    for ( Rank u = firstNbr(v); -1 < u; u = nextNbr(v, u) )

        switch ( status(u) )

            { /* ... 视u的状态分别处理 ... */ }

    status(v) = VISITED; //对v的访问结束

}

#undef hca
```

```
switch ( status(u) )
```

```
case UNDISCOVERED:
```

```
parent(u) = v; type(v, u) = TREE; //拓展树边
```

```
BCC( u, clock, S ); //从u开始遍历, 返回后...
```

```
if ( hca(u) < dTime(v) ) //若u经后向边指向v的真祖先
```

是否能够到更高节点祖先

```
hca(v) = min( hca(v), hca(u) ); //则v亦必如此
```

```
else //否则, 以v为关节点 (u以下即是一个BCC, 且其中顶点此时正集中于栈S的顶部)
```

```
while ( u != S.pop() ); //弹出当前BCC中 (除v外) 的所有节点
```

```
//可视需要做进一步处理
```

```
break;
```

v 可能作为多个关节点



```
switch ( status(u) )
```

```
case DISCOVERED:
```

```
    type(v, u) = BACKWARD;
```

```
    if ( u != parent(v) )
```

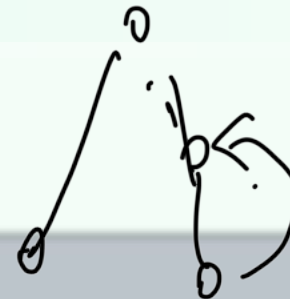
```
        hca(v) = min( hca(v), dTime(u) ); //更新hca(v), 越小越高
```

```
    break;
```

```
default: //VISITED (digraphs only)
```

```
    type(v, u) = dTime(v) < dTime(u) ? FORWARD : CROSS;
```

```
    break;
```



前向

跨边

复杂度

❖ 运行时间与常规的DFS相同，也是 $\mathcal{O}(n + e)$

自行验证：栈操作的复杂度也不过如此

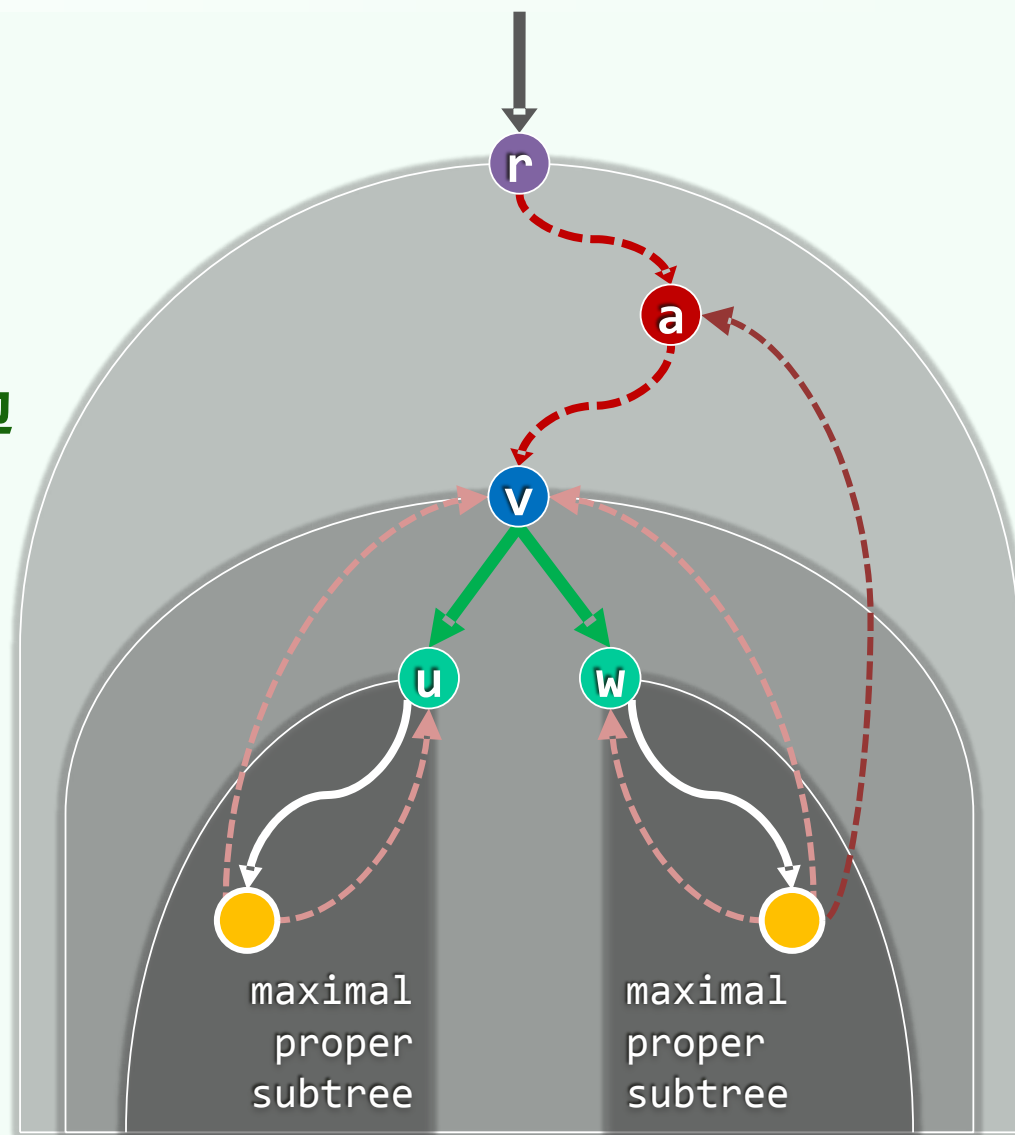
❖ 除原图本身，还需一个容量为 $\mathcal{O}(e)$ 的栈存放已访问的边

为支持递归，另需一个容量为 $\mathcal{O}(n)$ 的运行栈

❖ 如何推广至有向图的**强连通分量**

(Strongly-connected component)

- Kosaraju's algorithm
- Tarjan's algorithm



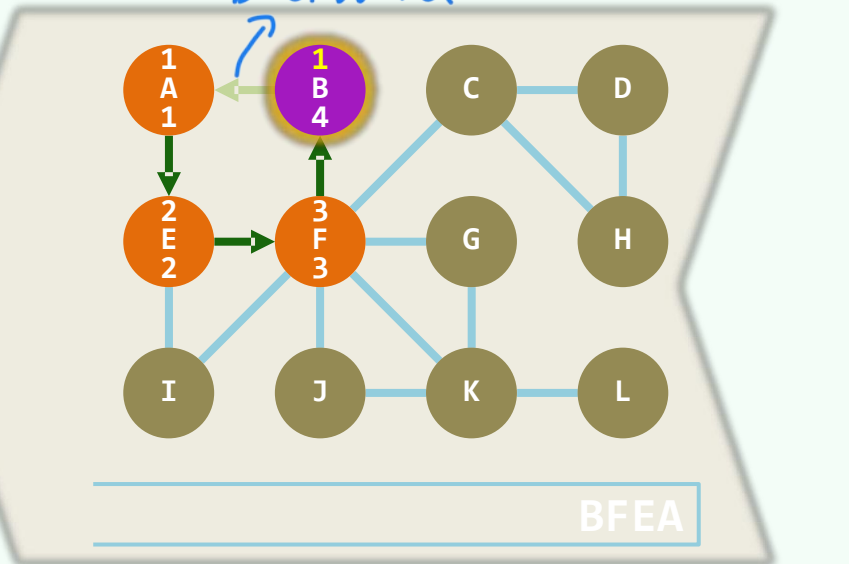
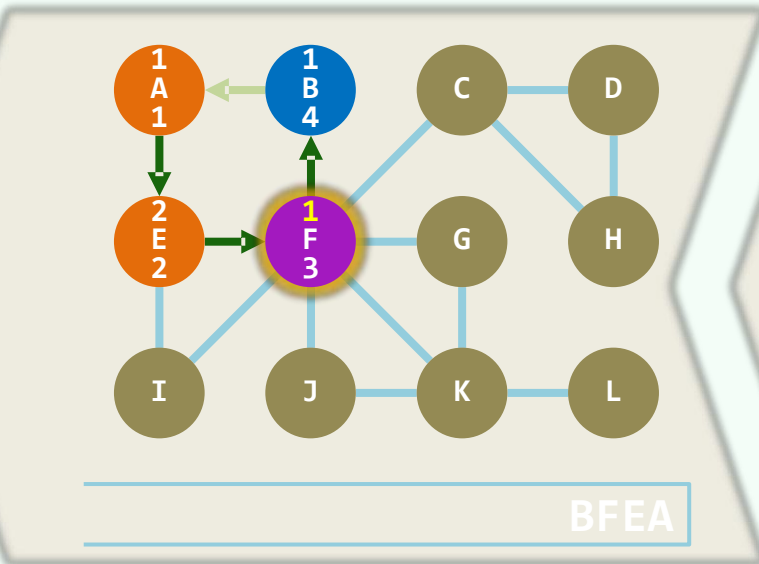
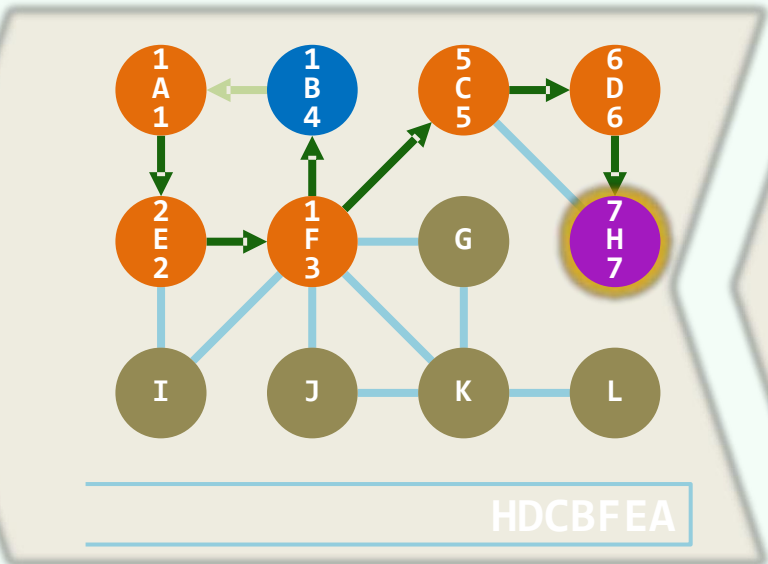
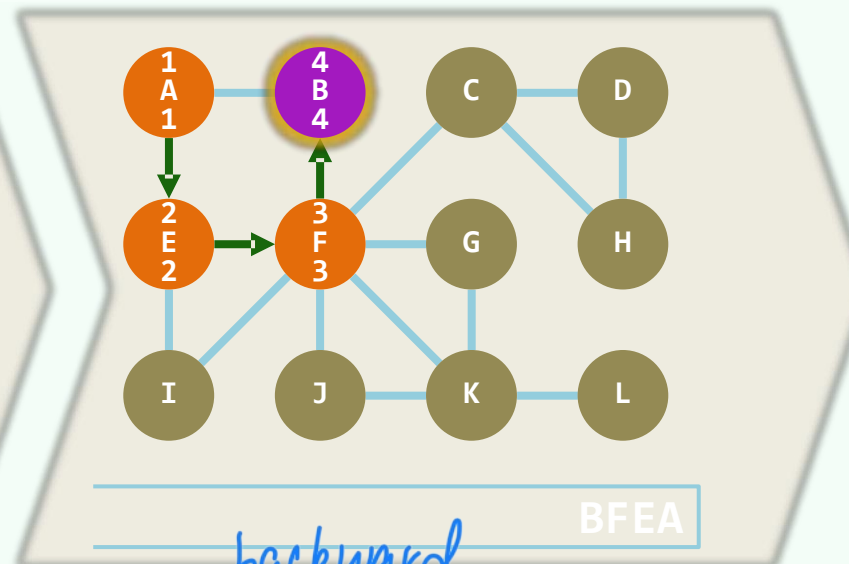
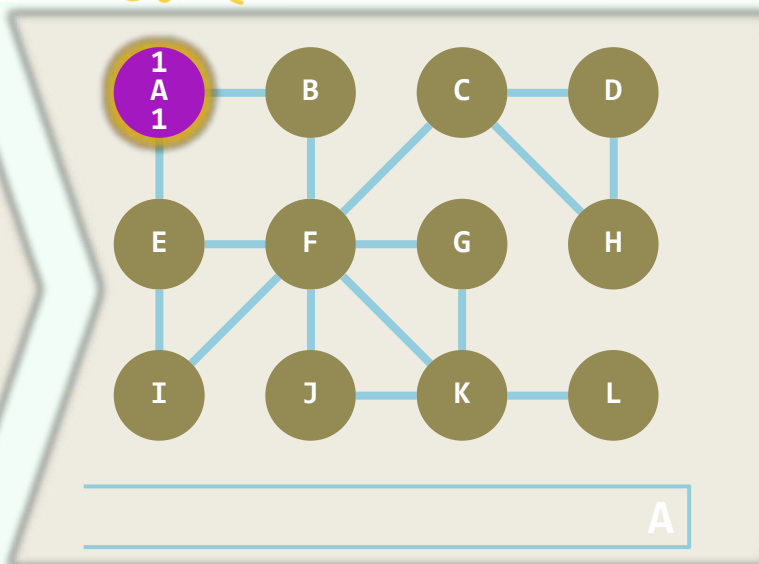
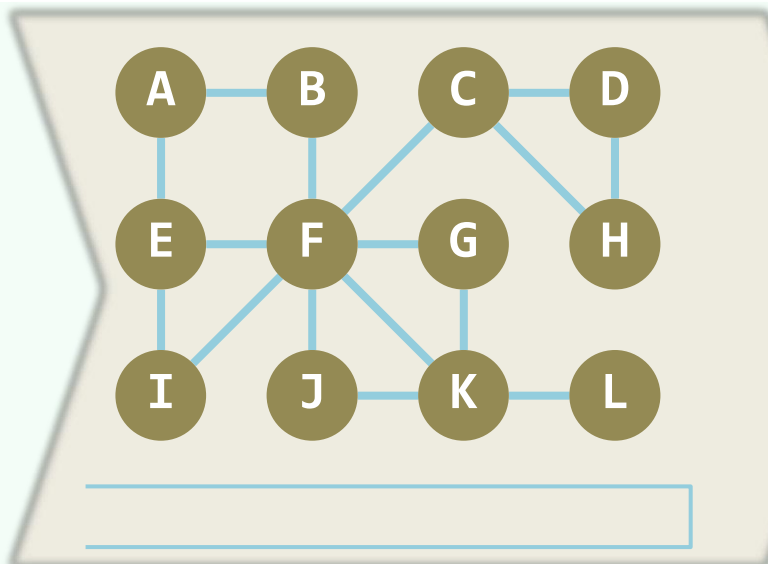
图应用

双联通分量：实例

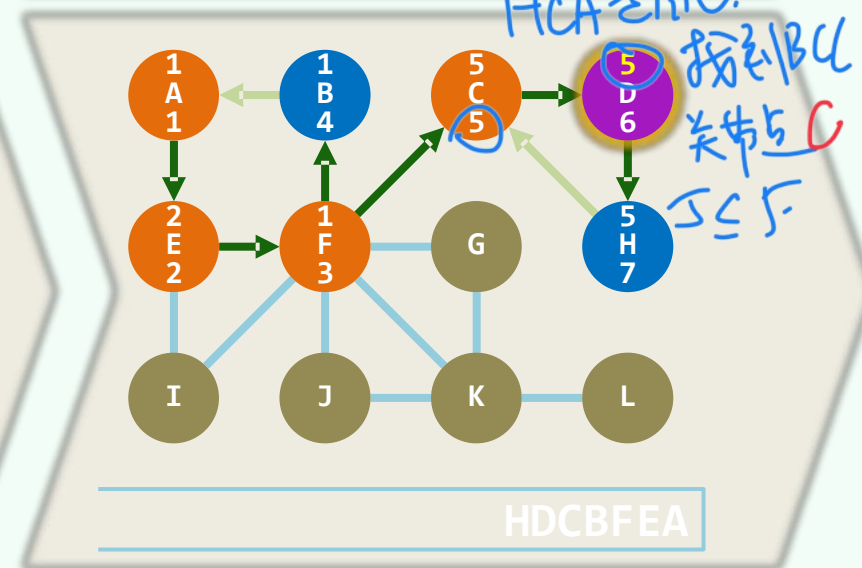
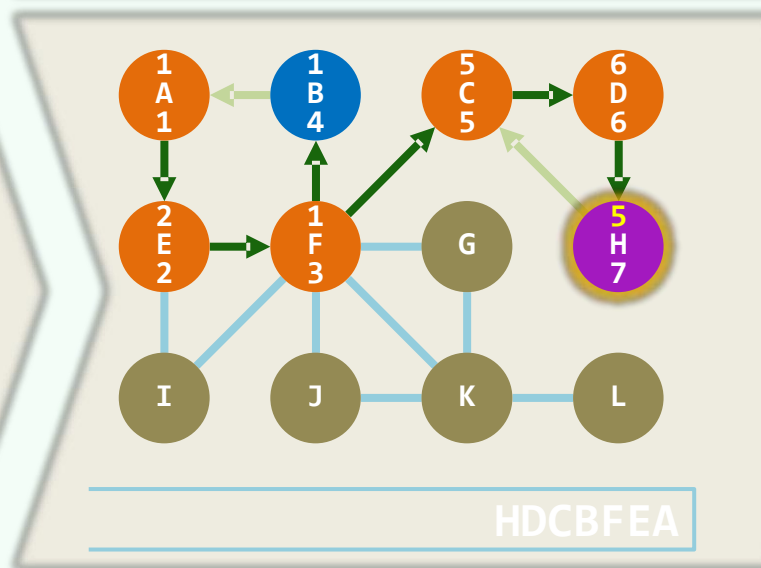
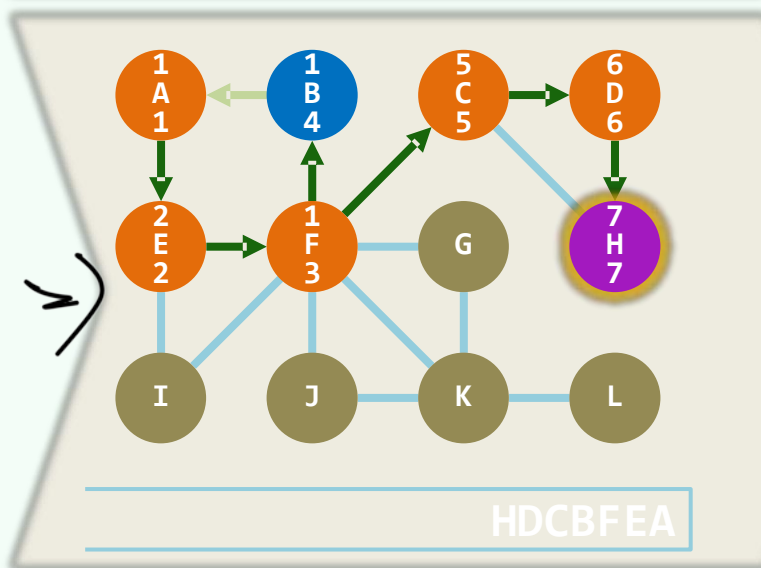
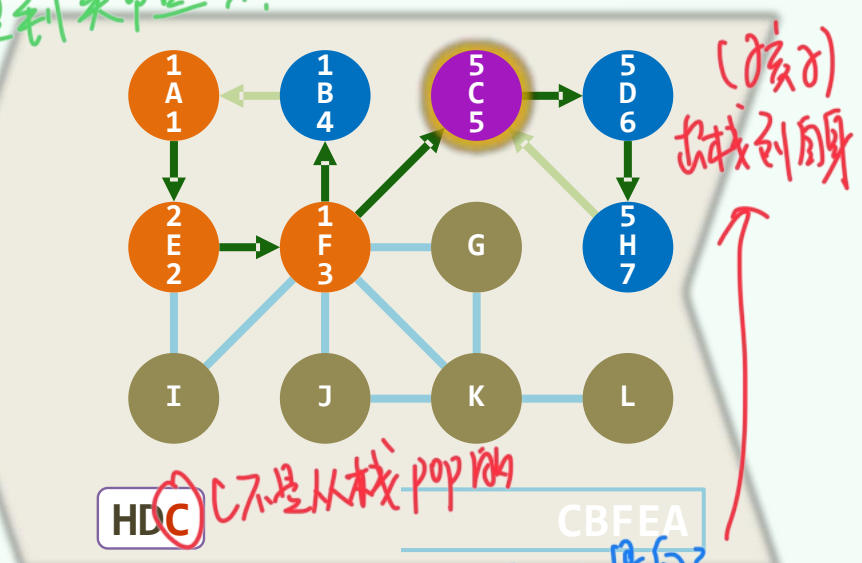
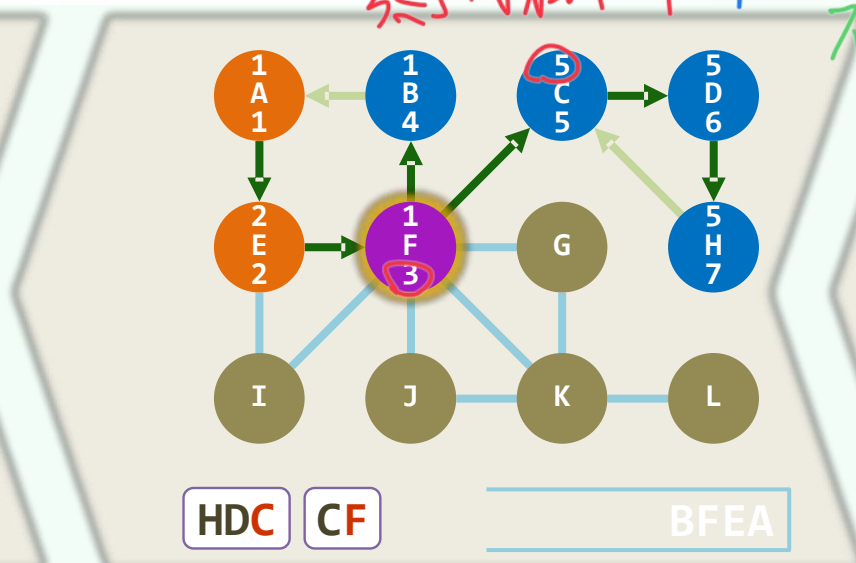
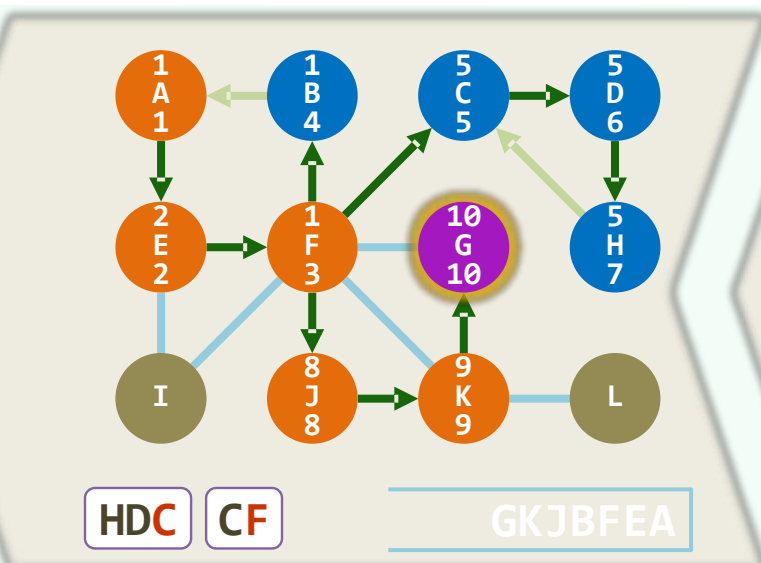
BCC: Example (1/5)

HCA
dtime

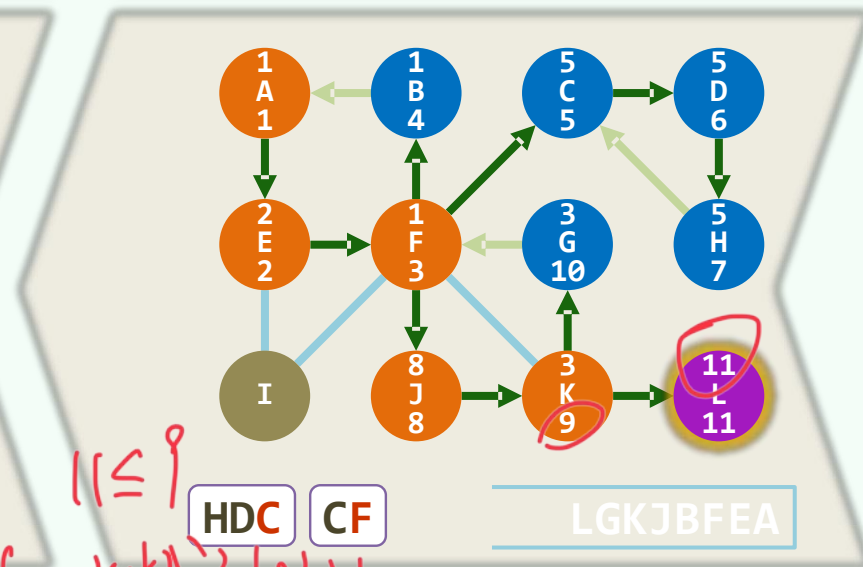
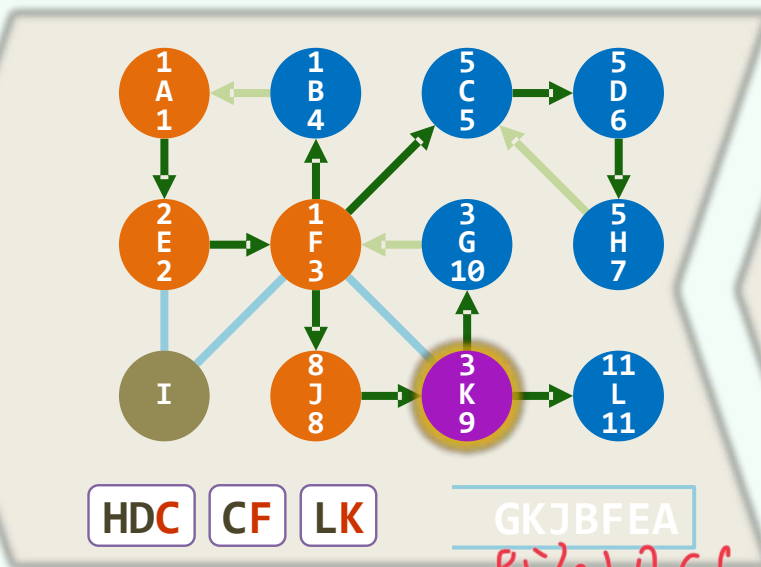
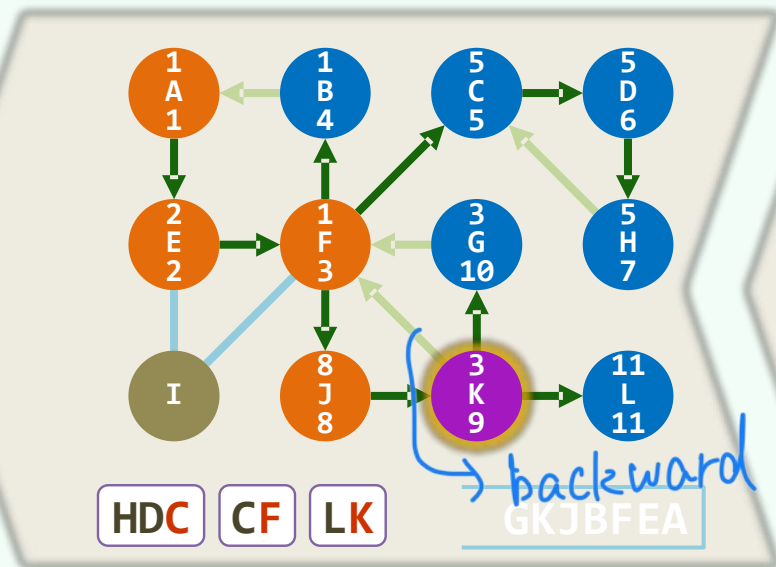
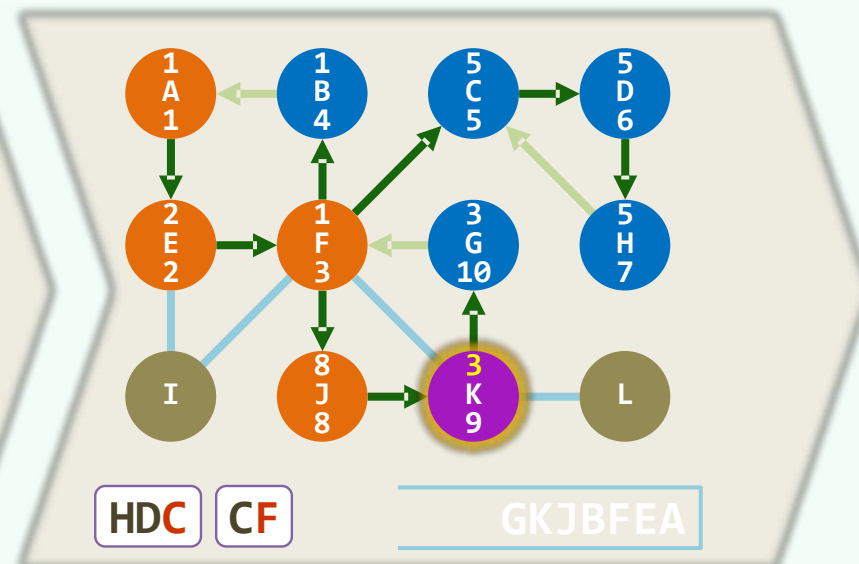
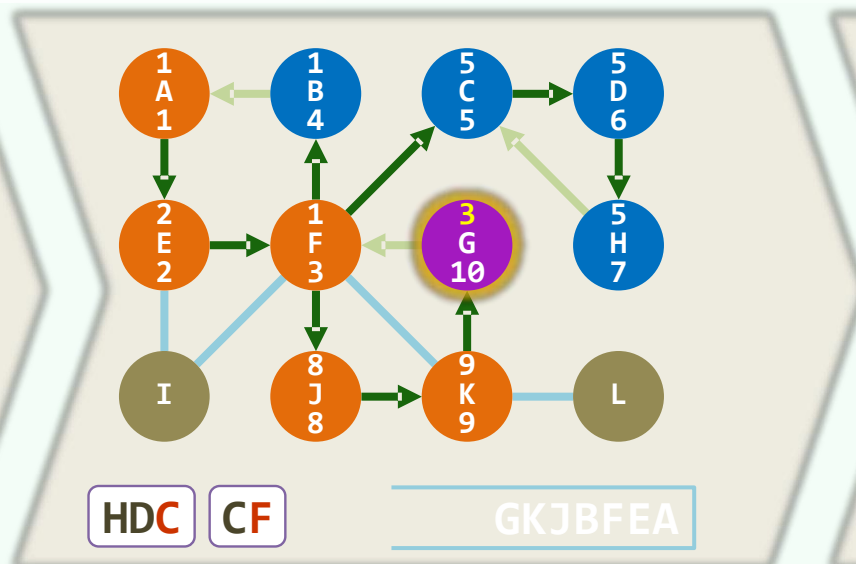
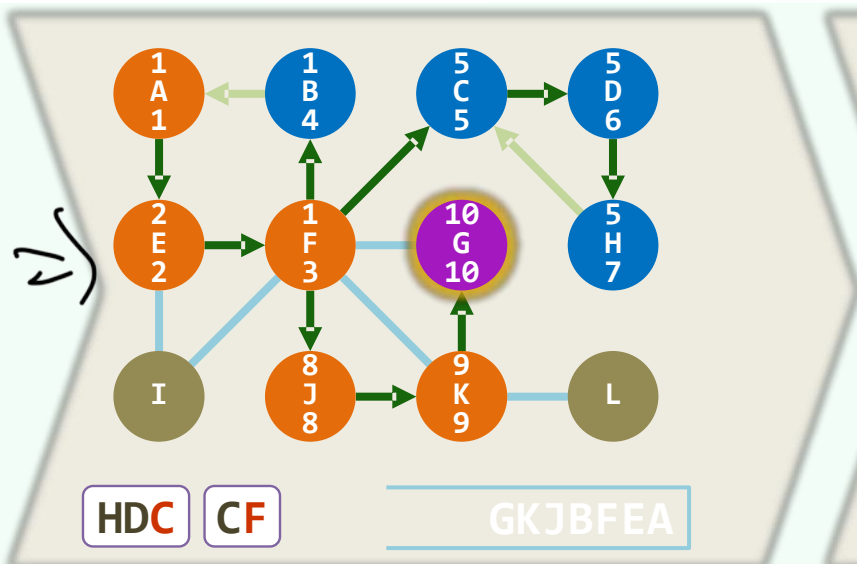
比HCA $\leq$ dtime, 说明能够更高祖先



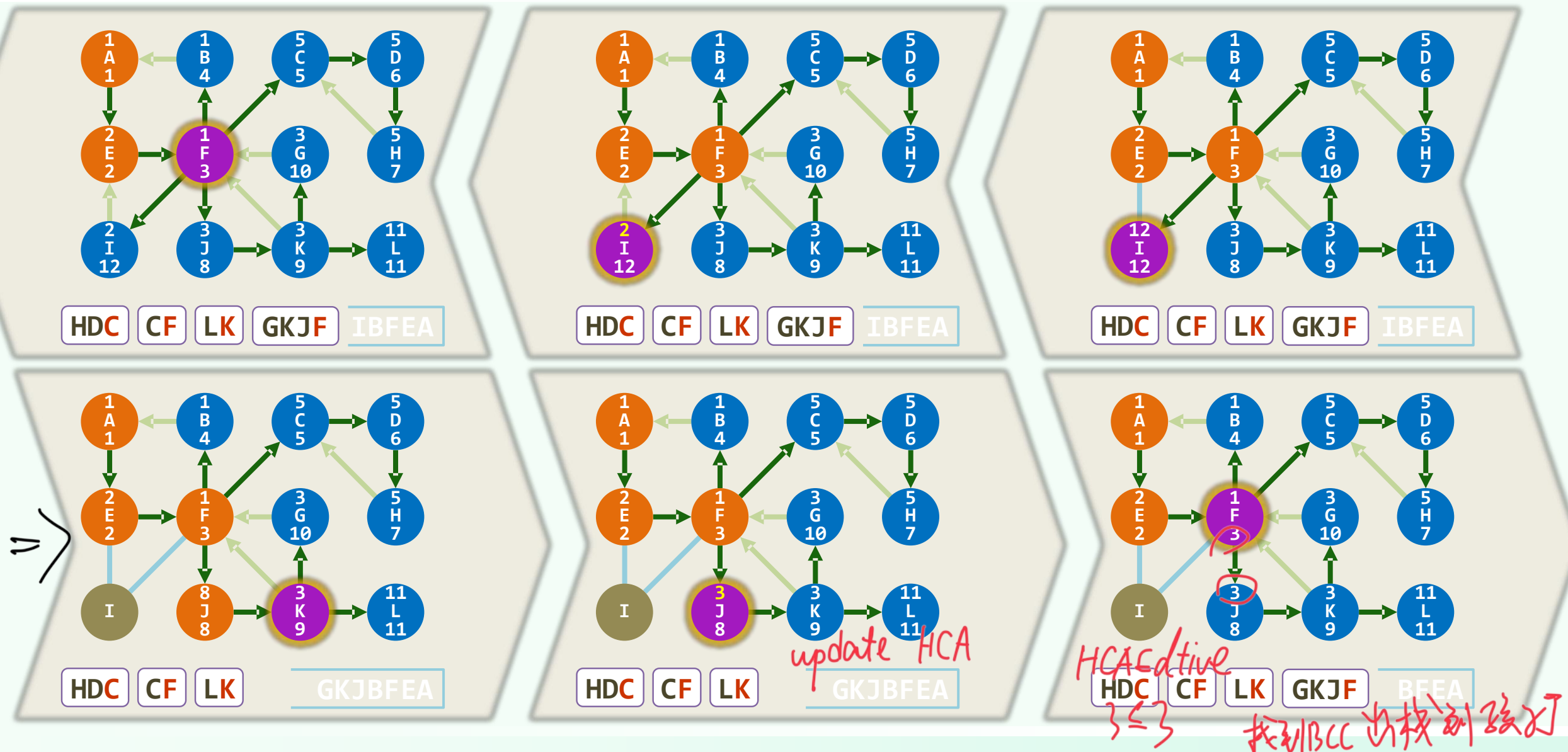
BCC: Example (2/5)



BCC: Example (3/5)



BCC: Example (4/5)



BCC: Example (5/5)

